



State



Learning Objectives

- Explain the method signature of state hook **useState**
- Build a component that uses a **useState** hook to keep component state.
- Explain that an **onClick** attribute in a react component creates an **event listener** for the click event
- Build a component with an **onClick** attribute that has a value of the function to invoke when the event is triggered
- **Model** the data a react application needs and create state hooks accordingly.



Intro

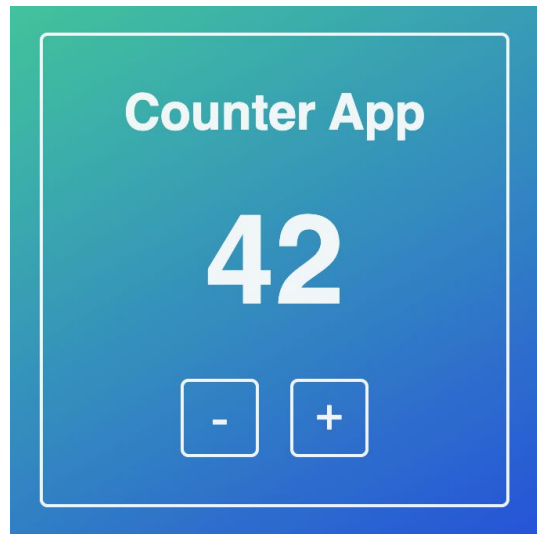
Here are some requirements and a mockup from a designer for a simple counter app.

A user

- Sees a title "Counter App"
- Sees the current counter count
- Can click on a minus sign to decrease the count by 1
- Can click on plus sign to increase the count by 1

Q: After understanding the requirements, in the modelling phase, what **state** does this app need?

Q: What does a user interaction look like?





Time for some live coding

Let's look at how this app could be built using plain JS



Time for some live coding

Now let's try and build the same app, but with React..

(An existing version [is available here](#))



useState

```
import { useState } from 'react'
```

To give our component some **state**, first, we must **import useState**



useState

```
const [count, setCount] = useState(0)
```

useState is a function that React gives to us. It's a special type of function React calls a "hook", but for now we can just think of it as a function.

Q: What's the signature of this function?



useState

```
const [count, setCount] = useState(0)
```

It takes: Our initial state

It returns: An array with 2 elements; the **current state**, and a **function to change that state !** This last part is **really important**. We don't change the state directly, we must use the function that useState returns.



useState

```
const [count, setCount] = useState(0)
```



The longer way

```
const countStateArray = useState(0)
const count = countStateArray[0]
const setCount = countStateArray[1]
```

Hold up! We have some **new syntax** here! This is called a **destructuring assignment**.

What this code does is declare 2 new const variables, **count** and **setCount**, and assign them the first and second values from the array returned from calling the **useState** function



useState

```
<h2 className='count'>{count}</h2>
```

Within the JSX our function returns, we can now **interpolate** our **current state** – the **count** variable – to display it. This means that the JS expression will get evaluated **before** the element renders. The h2 element renders with its content – the result of the expression. In this case it will be a number!



useState

```
<button className='down-button' onClick={() => setCount(count - 1)}>  
  -  
</button>  
<button className='up-button' onClick={() => setCount(count + 1)}>  
  +  
</button>
```

Finally, instead of having to call `addEventListener`, we give our buttons an **onClick attribute**, with the **function** we want to call as a **value**



useState

```
<button className='down-button' onClick={() => setCount(count - 1)}>  
  -  
</button>  
  
<button className='up-button' onClick={() => setCount(count + 1)}>  
  +  
</button>
```

Notice how our event listener functions can be defined inlined, just like with **addEventListener**

And just like `addEventListener`, these functions will receive an **Event**, but here we don't need it, so we **ignore it**



useState

```
const up = () => setCount(count + 1)
const down = () => setCount(count - 1)

<button className='down-button' onClick={down}>
  -
</button>
<button className='up-button' onClick={up}>
  +
</button>
```

Remember! This is passing
a *reference* to a function,
not calling the function

To make things cleaner, we could also create these functions **before the JSX**
And then just pass those functions as the values of the **onClick**



Time for some practice

Use vite to scaffold a new project – call the project “Counter App”

- Vite comes with a template counter – remove the code in the functional component in `src/App.jsx`
- You won't learn if you just use their code!

Build your own counter. A user should be able to:

- See a title “Counter App”
- See the current counter count
- Click on a minus sign to decrease the count by 1
- Click on plus sign to increase the count by 1

Extensions

- A user can't increase the count beyond 10 or decrease it below 0
- A user can click a “double” button to double the value of the counter
- A user can click a “halve” button to halve the counter. The value should be rounded to the nearest whole number
- A user sees the font-size of the counter increase as it approaches 10, and decrease as it approaches 0



map



Modelling State

Now let's look at a slightly more involved app

Q: What state does this app need to work?

OPTIONS

Show completed ☒

ADD ITEM

Add

TODO

☐ Go shopping 

☐ Work out 

COMPLETED

☒ See the doctor 



Time for some live coding

Let's look at how we previously built this app using plain JS



Time for some live coding

Now let's look at the same app, but built using React



State types

```
const [todos, setTodos] = useState([])
const [todoInput, setTodoInput] = useState('')
const [showCompletedTodos, setShowCompletedTodos] = useState(true)
```

Notice also how we can store **any valid JS data type** as state



External variables

```
const initialTodos = [
  {
    text: 'Go shopping',
    completed: false
  },
  ...
]

export default function App() {
  const [todos, setTodos] = useState(initialTodos)
  ...
}
```

Note that we can create **variables outside our components**, too

These are useful especially when we have some **data that won't change over time**



Time for some practice

Go back to the Breweries Tours challenge. What state would you need to model for this app in React?

No code required!

Outcome: a diagram, sketch or note of the application state needed and how this would be handled in React.